

ChatGPT - OS

what are the unix system call using the #include "unistd.h" , and explain what it is use of api in program

The header file:

```
C
#include <unistd.h>
```

is one of the **most important header files in UNIX/Linux system programming**. It declares many **POSIX (Portable Operating System Interface)** APIs and UNIX **system calls** that allow a program to communicate with the operating system kernel.

Your system programming assignments include many of these APIs such as **fork()**, **getpid()**, **exec()**, **pipe()**, **read()**, **write()**, **close()**, etc.

SYSTEM PROGRAMMING ASSESSMENT Process_Threads_Semaphore_Mutex...

What is unistd.h?

unistd.h stands for

UNIX Standard Header

It contains the **function prototypes** (declarations) for many UNIX/POSIX system calls.

Without this header, the compiler does not know the return type and parameters of these functions.

Example:

```
C
#include <unistd.h>

int main()
{
    fork();
}
```

Here,

```
C
#include <unistd.h>
```

tells the compiler that

```
C
pid_t fork(void);
```

exists.

Why do we need `unistd.h`?

Suppose you write

```
C
fork();
```

The compiler asks:

What is `fork()`?

What does it return?

What parameters does it take?

The answers are inside

```
C
unistd.h
```

Without it,

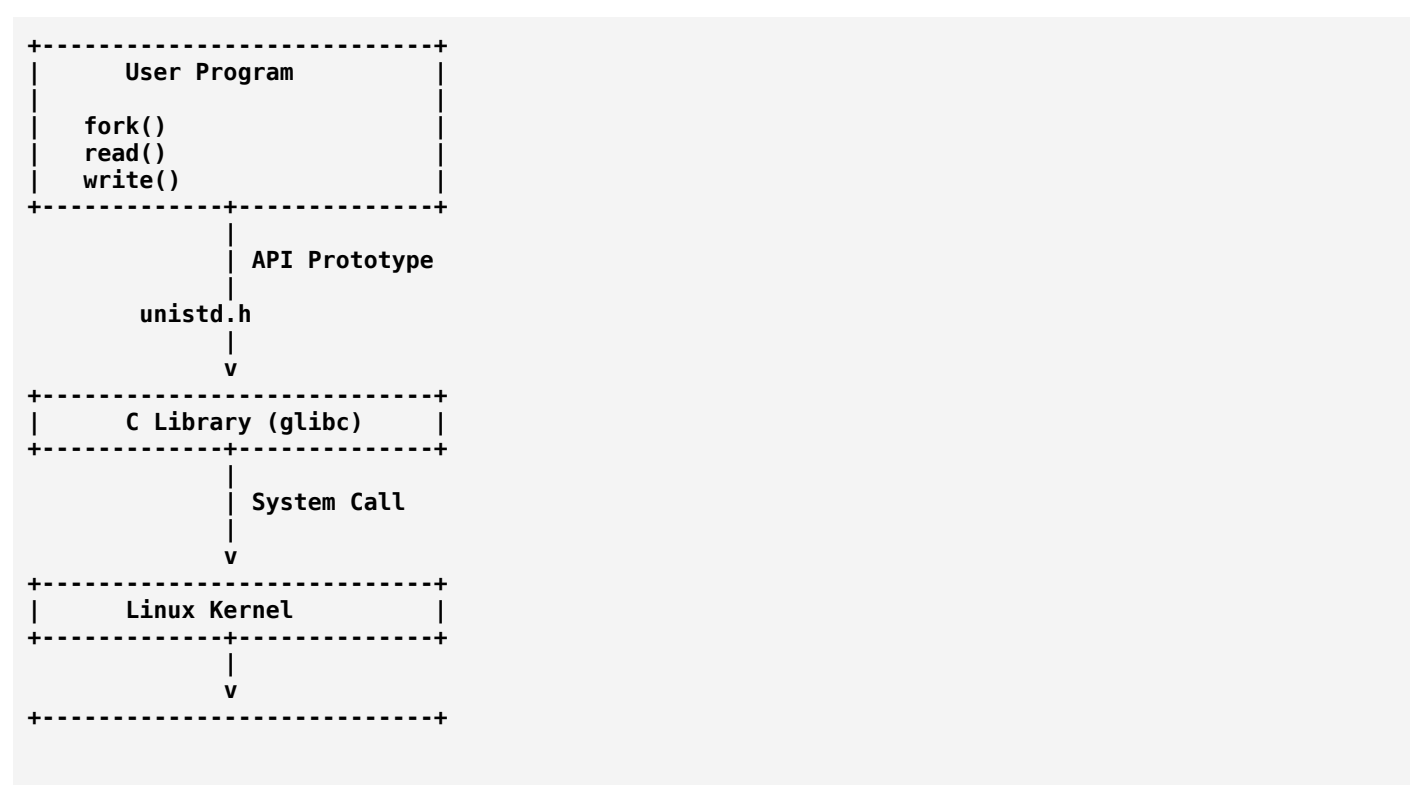
```
warning: implicit declaration of function 'fork'
```

or

```
error: unknown function fork()
```

may occur.

How a System Call Works



| Hardware (CPU, Memory...) |
+-----+

Notice that `unistd.h` **itself does not implement the function**. It only provides the declaration. The actual implementation is in the C library, which invokes the kernel system call.

Major UNIX APIs in `unistd.h`

The APIs can be grouped according to their purpose.

1. Process Management APIs

These create and manage processes.

API	Purpose	Return Type
<code>fork()</code>	Create a child process	<code>pid_t</code>
<code>getpid()</code>	Get current process ID	<code>pid_t</code>
<code>getppid()</code>	Get parent process ID	<code>pid_t</code>
<code>exec()</code> family	Replace current process image	varies
<code>_exit()</code>	Terminate process immediately	void
<code>alarm()</code>	Schedule a signal after N seconds	unsigned int
<code>sleep()</code>	Suspend execution	unsigned int
<code>pause()</code>	Wait until a signal arrives	int

`fork()`

Prototype

```
C
pid_t fork(void);
```

Purpose

Creates an exact copy of the current process.

Example

```
C
pid_t pid;
pid = fork();
if(pid == 0)
{
    printf("Child\n");
}
else
{
    }
```

```
printf("Parent\n");  
}
```

getpid()

Prototype

```
C  
pid_t getpid(void);
```

Returns

Current process ID.

Example

```
C  
printf("%d", getpid());
```

Output

```
1254
```

getppid()

Prototype

```
C  
pid_t getppid(void);
```

Returns

Parent process ID.

Example

```
C  
printf("%d", getppid());
```

sleep()

Prototype

```
C  
unsigned int sleep(unsigned int seconds);
```

Purpose

Pauses execution.

Example

```
C
printf("Start");
sleep(5);
printf("End");
```

Output

```
Start
(wait 5 seconds)
End
```

2. File Management APIs

These work with files using **file descriptors**.

API	Purpose
<code>read()</code>	Read bytes
<code>write()</code>	Write bytes
<code>close()</code>	Close file
<code>lseek()</code>	Change file position
<code>unlink()</code>	Delete file
<code>access()</code>	Check permissions
<code>fsync()</code>	Flush file buffers
<code>isatty()</code>	Check terminal

read()

Prototype

```
C
ssize_t read(int fd,
             void *buffer,
             size_t count);
```

Example

```
C
char data[100];
read(fd, data, 100);
```

Meaning

Read
100 bytes
from file descriptor fd
into buffer

write()

Prototype

```
C
ssize_t write(int fd,
              const void *buffer,
              size_t count);
```

Example

```
C
write(1, "Hello", 5);
```

Output

Hello

Here

1

means

STDOUT

close()

Prototype

```
C
int close(int fd);
```

Purpose

Closes a file descriptor.

Example

```
C
close(fd);
```

lseek()

Prototype

```
C
off_t lseek(int fd,
            off_t offset,
            int whence);
```

Purpose

Moves file pointer.

Example

```
C
lseek(fd, 0, SEEK_SET);
```

Moves pointer to beginning.

3. Pipe APIs

Used for inter-process communication (IPC).

API	Purpose
pipe()	Create unnamed pipe

Prototype

```
C
int pipe(int fd[2]);
```

Example

```
C
int fd[2];
pipe(fd);
```

Creates

```
fd[0] -> Read end
fd[1] -> Write end
```

4. Directory APIs

API	Purpose
chdir()	Change directory
getcwd()	Get current directory

API	Purpose
<code>rmdir()</code>	Remove directory

getcwd()

Prototype

```
C
char *getcwd(char *buf,
              size_t size);
```

Example

```
C
char path[100];
getcwd(path, 100);
printf("%s", path);
```

Output

```
/home/student
```

chdir()

Prototype

```
C
int chdir(const char *path);
```

Example

```
C
chdir("/home/user");
```

Changes working directory.

5. User Information APIs

API	Purpose
<code>getuid()</code>	User ID
<code>geteuid()</code>	Effective User ID
<code>getgid()</code>	Group ID
<code>getegid()</code>	Effective Group ID

Example

```
C
printf("%d", getuid());
```

6. Terminal APIs

API	Purpose
<code>isatty()</code>	Check terminal
<code>ttyname()</code>	Get terminal name

Example

```
C
if(isatty(0))
{
    printf("Keyboard Input");
}
```

7. Symbolic Link APIs

API	Purpose
<code>link()</code>	Create hard link
<code>symlink()</code>	Create symbolic link
<code>readlink()</code>	Read symbolic link

8. File Permission APIs

API	Purpose
<code>access()</code>	Check permission
<code>unlink()</code>	Delete file

Example

```
C
if(access("data.txt", F_OK)==0)
{
    printf("File exists");
}
```

9. Miscellaneous APIs

API	Purpose
<code>dup()</code>	Duplicate file descriptor

API	Purpose
<code>dup2()</code>	Duplicate to specified descriptor
<code>sync()</code>	Flush filesystem buffers
<code>fsync()</code>	Flush file
<code>truncate()</code>	Resize file

Common APIs from `unistd.h` Used in System Programming

API	Category	Purpose
<code>fork()</code>	Process	Create child process
<code>getpid()</code>	Process	Current process ID
<code>getppid()</code>	Process	Parent process ID
<code>exec*()</code>	Process	Replace current process
<code>_exit()</code>	Process	Exit immediately
<code>sleep()</code>	Process	Delay execution
<code>alarm()</code>	Signal	Set alarm
<code>pause()</code>	Signal	Wait for signal
<code>read()</code>	File	Read data
<code>write()</code>	File	Write data
<code>close()</code>	File	Close file
<code>lseek()</code>	File	Move file pointer
<code>pipe()</code>	IPC	Create unnamed pipe
<code>dup()</code>	File	Duplicate descriptor
<code>dup2()</code>	File	Redirect descriptor
<code>getcwd()</code>	Directory	Current working directory
<code>chdir()</code>	Directory	Change directory
<code>access()</code>	File	Check permissions
<code>unlink()</code>	File	Delete file
<code>link()</code>	File	Create hard link
<code>symlink()</code>	File	Create symbolic link
<code>readlink()</code>	File	Read symbolic link
<code>getuid()</code>	User	Get user ID
<code>geteuid()</code>	User	Get effective user ID
<code>getgid()</code>	User	Get group ID
<code>getegid()</code>	User	Get effective group ID
<code>isatty()</code>	Terminal	Check terminal device
<code>ttyname()</code>	Terminal	Get terminal name

APIs in `unistd.h` Used in Your Assignments

Based on the uploaded system programming assignment and notes, the following `unistd.h` APIs are directly relevant:

API	Used For
<code>fork()</code>	Process creation
<code>getpid()</code>	Display process ID
<code>getppid()</code>	Display parent process ID
<code>exec()</code> family (<code>execl()</code> , <code>execv()</code> , <code>execvp()</code>)	Execute another program or command
<code>pipe()</code>	Unnamed pipe IPC
<code>read()</code>	Read from files or pipes
<code>write()</code>	Write to files or pipes
<code>close()</code>	Close file descriptors
<code>sleep()</code>	Introduce delays for synchronization or demonstration
<code>_exit()</code>	Child process termination
<code>lseek()</code>	Random file access
<code>dup()</code> / <code>dup2()</code>	File descriptor duplication and redirection

These functions appear throughout the process management, file management, and pipe programming exercises in your uploaded materials.

SYSTEM PROGRAMMING ASSESSMENT SYSTEM PROGRAMMING ASSESSMENT

Process_Threads_Semaphore_Mutex...

Final Summary

- `#include <unistd.h>` is the standard POSIX header that declares many UNIX/Linux system call interfaces.
- It provides **API prototypes**, not the implementations.
- Programs use these APIs to request services from the Linux kernel, such as process creation, file I/O, inter-process communication, directory handling, and user information.
- For system programming in Linux, it is one of the most frequently included headers, alongside headers such as `sys/types.h`, `sys/wait.h`, `fcntl.h`, `pthread.h`, and `sys/stat.h`.

In your coursework, the most important `unistd.h` APIs to master are: `fork()`, `exec()` family, `getpid()`, `getppid()`, `read()`, `write()`, `close()`, `pipe()`, `dup()`, `dup2()`, `lseek()`, `sleep()`, and `_exit()`. These form the foundation for process management, file handling, and IPC in UNIX/Linux.

for the process management give that all apis , with out miss of any one

If you are studying **UNIX/Linux Process Management** for **System Programming**, then the APIs are spread across several header files, not just `unistd.h`. Below is a **complete list of process management**

APIs commonly available in POSIX/Linux. This covers virtually everything you will encounter in B.E./B.Tech, interviews, and Linux system programming.

Process Management Header Files

```
C
#include <unistd.h>      // Process creation, exec, IDs
#include <sys/types.h>    // pid_t
#include <sys/wait.h>     // wait(), waitpid(), waitid()
#include <signal.h>       // kill(), signal handling
#include <sched.h>        // Scheduling APIs
#include <sys/resource.h> // Priority/resource limits
#include <time.h>         // nanosleep(), clock APIs
```

1. Process Creation APIs

These APIs create new processes.

API	Prototype	Header	Purpose
fork()	pid_t fork(void);	unistd.h	Create child process
vfork()	pid_t vfork(void);	unistd.h	Create child sharing parent's address space until exec/exit
clone() (Linux-specific)	int clone(...);	sched.h	Create process/thread with selectable resource sharing

1. fork()

```
C
pid_t fork(void);
```

Creates a duplicate process.

Returns

```
0      → Child
>0     → Parent (child PID)
-1     → Error
```

2. vfork()

```
C
pid_t vfork(void);
```

Similar to **fork()**, but:

- Child shares parent's memory.

- Parent is suspended until child calls `exec()` or `_exit()`.
- Faster than `fork()`.

3. `clone()`

```
C
int clone(...);
```

Linux kernel API.

Used internally for:

- Threads
- Containers
- Namespaces

2. Process Identification APIs

These identify the current process, its parent, and related information.

API	Purpose
<code>getpid()</code>	Current process ID
<code>getppid()</code>	Parent process ID
<code>getuid()</code>	Real user ID
<code>geteuid()</code>	Effective user ID
<code>getgid()</code>	Group ID
<code>getegid()</code>	Effective group ID
<code>getpgrp()</code>	Process group ID
<code>getpgid()</code>	Process group of specified PID
<code>getsid()</code>	Session ID

`getpid()`

```
C
pid_t getpid(void);
```

Returns

Current Process ID

getppid()

```
C  
pid_t getppid(void);
```

Returns

Parent Process ID

getuid()

```
C  
uid_t getuid(void);
```

Returns real user ID.

geteuid()

```
C  
uid_t geteuid(void);
```

Returns effective user ID.

getgid()

```
C  
gid_t getgid(void);
```

Returns group ID.

getegid()

```
C  
gid_t getegid(void);
```

Returns effective group ID.

getpgrp()

```
C
```

```
pid_t getpgrp(void);
```

Returns process group ID.

getpgid()

```
C
pid_t getpgid(pid_t pid);
```

Returns process group of another process.

getsid()

```
C
pid_t getsid(pid_t pid);
```

Returns session ID.

3. Process Replacement APIs (Exec Family)

These replace the current process image with another program.

API	Purpose
execl()	Execute using argument list
execvp()	Search executable in PATH
execle()	Execute with custom environment
execv()	Execute using argument array
execvp()	Execute using PATH and array
execvpe() (GNU)	PATH + environment
fexecve()	Execute using file descriptor
execve()	Kernel-level exec

All exec() Family

```
C
execl()
execvp()
execle()

execv()
execvp()
execvpe()
```

```
execve()
fexecve()
```

Purpose

```
Replace current process
Old program removed
New program loaded
```

4. Process Termination APIs

API	Purpose
<code>exit()</code>	Normal termination (C library)
<code>_exit()</code>	Immediate termination
<code>_Exit()</code>	ISO C immediate exit
<code>abort()</code>	Abnormal termination

`_exit()`

```
C
void _exit(int status);
```

Immediately terminates process.
No flushing of stdio buffers.

`exit()`

```
C
void exit(int status);
```

Flushes buffers.
Calls cleanup handlers.

`abort()`

```
C
void abort(void);
```

Terminates process and raises **SIGABRT**.

5. Waiting APIs

Used by the parent process to wait for child processes.

API	Purpose
<code>wait()</code>	Wait for any child
<code>waitpid()</code>	Wait for specific child
<code>waitid()</code>	Advanced waiting
<code>wait3()</code> (BSD)	Wait with resource usage
<code>wait4()</code> (BSD/Linux)	Wait with resource usage

`wait()`

```
C
pid_t wait(int *status);
```

Waits until one child finishes.

`waitpid()`

```
C
pid_t waitpid(pid_t pid,
              int *status,
              int options);
```

Waits for specific child.

`waitid()`

```
C
int waitid(...);
```

Advanced waiting API.

6. Signal APIs

Signals control processes asynchronously.

API	Purpose
<code>kill()</code>	Send signal
<code>raise()</code>	Send signal to self

API	Purpose
signal()	Install signal handler
sigaction()	Reliable signal handler
pause()	Wait for signal
alarm()	Generate signal after time
sleep()	Suspend process
nanosleep()	High-resolution sleep

kill()

```
C
int kill(pid_t pid,
         int signal);
```

Example

```
C
kill(pid, SIGKILL);
```

signal()

```
C
void (*signal(int sig,
              void (*handler)(int)))(int);
```

Registers signal handler.

sigaction()

```
C
int sigaction(...);
```

Modern replacement for **signal()**.

pause()

```
C
int pause(void);
```

Waits until signal arrives.

alarm()

```
C
unsigned int alarm(unsigned int seconds);
```

Generates SIGALRM.

sleep()

```
C
unsigned int sleep(unsigned int sec);
```

Sleep in seconds.

nanosleep()

```
C
int nanosleep(...);
```

Nanosecond resolution.

7. Scheduling APIs

API	Purpose
nice()	Change priority
sched_yield()	Yield CPU
sched_getscheduler()	Get scheduling policy
sched_setscheduler()	Set scheduling policy
sched_getparam()	Get priority
sched_setparam()	Set priority
sched_rr_get_interval()	Round-robin interval

8. Process Group APIs

API	Purpose
setpgid()	Set process group
setsid()	Create new session

API	Purpose
<code>tcgetpgrp()</code>	Terminal process group
<code>tcsetpgrp()</code>	Set terminal process group

9. Resource Management APIs

API	Purpose
<code>getpriority()</code>	Get priority
<code>setpriority()</code>	Set priority
<code>getrlimit()</code>	Resource limits
<code>setrlimit()</code>	Set limits
<code>getrusage()</code>	Resource usage

10. Miscellaneous Process APIs

API	Purpose
<code>system()</code>	Execute shell command
<code>daemon()</code>	Create daemon process
<code>gethostname()</code>	Get host name
<code>sethostname()</code>	Set host name (privileged)

APIs Commonly Used in System Programming Courses

These are the APIs most frequently used in university assignments and introductory system programming. They also align with the process-management tasks in your uploaded assignment document.

SYSTEM PROGRAMMING ASSESSMENT

API	Header	Description
<code>fork()</code>	<code>unistd.h</code>	Create child process
<code>vfork()</code>	<code>unistd.h</code>	Lightweight process creation
<code>getpid()</code>	<code>unistd.h</code>	Get process ID
<code>getppid()</code>	<code>unistd.h</code>	Get parent process ID
<code>execl()</code>	<code>unistd.h</code>	Execute new program
<code>execvp()</code>	<code>unistd.h</code>	Execute using PATH
<code>execle()</code>	<code>unistd.h</code>	Execute with environment
<code>execv()</code>	<code>unistd.h</code>	Execute using argument array
<code>execvp()</code>	<code>unistd.h</code>	Execute using PATH and array

API	Header	Description
<code>execve()</code>	<code>unistd.h</code>	Kernel-level exec
<code>_exit()</code>	<code>unistd.h</code>	Immediate process termination
<code>wait()</code>	<code>sys/wait.h</code>	Wait for any child
<code>waitpid()</code>	<code>sys/wait.h</code>	Wait for specific child
<code>kill()</code>	<code>signal.h</code>	Send signal
<code>signal()</code>	<code>signal.h</code>	Install signal handler
<code>sigaction()</code>	<code>signal.h</code>	Advanced signal handling
<code>sleep()</code>	<code>unistd.h</code>	Delay execution
<code>nanosleep()</code>	<code>time.h</code>	High-resolution delay
<code>alarm()</code>	<code>unistd.h</code>	Generate timed signal
<code>pause()</code>	<code>unistd.h</code>	Wait for signal
<code>getuid()</code>	<code>unistd.h</code>	Get user ID
<code>geteuid()</code>	<code>unistd.h</code>	Get effective user ID
<code>getgid()</code>	<code>unistd.h</code>	Get group ID
<code>getegid()</code>	<code>unistd.h</code>	Get effective group ID
<code>getpgrp()</code>	<code>unistd.h</code>	Get process group ID
<code>setpgid()</code>	<code>unistd.h</code>	Set process group
<code>getsid()</code>	<code>unistd.h</code>	Get session ID
<code>setsid()</code>	<code>unistd.h</code>	Create new session

Final Note

This list includes the **complete set of standard POSIX/Linux process management APIs** that are typically covered in operating systems and system programming courses. For your current assignments, the core APIs you should master first are:

- Process creation: `fork()`, `vfork()`
- Process identification: `getpid()`, `getppid()`
- Process replacement: the entire `exec()` family
- Process synchronization: `wait()`, `waitpid()`
- Process termination: `exit()`, `_exit()`
- Signal handling: `kill()`, `signal()`, `sigaction()`
- Timing: `sleep()`, `alarm()`, `pause()`

These form the foundation for UNIX/Linux process management.

what is the use of `exec()` explain with program level , what it return

`exec()` is one of the **most important UNIX/Linux system calls**. It is also one of the most misunderstood by beginners because **it does not create a new process**.

Many students think:

`exec()` creates another process.

This is incorrect.

The correct understanding is:

`exec()` **replaces the current running program with another program while keeping the same process.**

What is `exec()`?

`exec()` is a family of system calls that **loads a new executable program into the current process memory**.

Prototype (kernel version):

```
C
int execve(const char *pathname,
           char *const argv[],
           char *const envp[]);
```

All other `exec()` functions (`execl()`, `execv()`, `execvp()`, etc.) are wrappers around `execve()`.

Header

```
C
#include <unistd.h>
```

Why do we need `exec()`?

Suppose a process is currently running:

```
Calculator Program
```

Now you want the same process to run

```
ls -l
```

instead.

Instead of creating another process, Linux **replaces** the calculator program with the `ls` program.

This is exactly what `exec()` does.

Important Point

`exec()` does **NOT**

- create a process
- duplicate a process
- copy memory

It **replaces** the currently running program.

Before and After exec()

Suppose

PID = 4500

Initially

Process 4500

```
+-----+
| main()
| printf()
| variables
| stack
| heap
+-----+
```

Now

```
C
execl("/bin/ls","ls","-l",NULL);
```

After execution

PID = 4500

```
+-----+
| ls Program
| new stack
| new heap
| new code
+-----+
```

Notice

PID remains SAME

Everything else changes.

Program Memory Before exec()

```
+-----+
| Text Segment
| (Your Program Code)
+-----+

| Global Variables
+-----+
```

```
| Heap |
+-----+

| Stack |
+-----+
```

After `exec()`

```
+-----+
| Text Segment |
| (ls Program) |
+-----+

| Global Variables |
+-----+

| Heap |
+-----+

| Stack |
+-----+
```

Everything belongs to the **new program**.

Real-Life Analogy

Imagine a classroom.

Student:

```
Roll No = 25
```

Teacher says

```
Replace your Mathematics book
with Physics book.
```

What changes?

```
Book changes
```

What remains?

```
Roll Number
```

Exactly the same happens in `exec()`.

```
PID = Roll Number
```

```
Program = Book
```

Basic Program

```
C
#include <stdio.h>
#include <unistd.h>
```



```
int main()
{
    printf("Before exec()\n");

    execl("/bin/ls", "ls", "-l", NULL);

    printf("After exec()\n");

    return 0;
}
```

Output

Before exec()

```
total 20
-rwxr-xr-x
...
```

Notice

After exec()

is **never printed**.

Why?

Because

```
main()
↓
printf()
↓
execl()
↓
Current program destroyed
↓
ls program starts
↓
Old program no longer exists
```

Step-by-Step Execution

Step 1

Program starts

```
main()
↓
printf()
```

Output

Before exec()

Step 2

Program executes

```
C
exec1("/bin/ls", "ls", "-l", NULL);
```

Kernel receives request

Replace this program.

Step 3

Kernel removes

```
main()
printf()
variables
stack
heap
```

Step 4

Kernel loads

```
/bin/ls
```

Step 5

Execution begins from

```
ls main()
```

not

```
our main()
```

Process Diagram

```
Initially
PID 4000
↓
main()
```

```

↓
printf()
↓
exec()
↓
Kernel
↓
Load ls
↓
ls main()
↓
Exit

```

Why Doesn't Control Return?

Because the old program no longer exists.

Suppose

```

Program A
↓
exec()
↓
Program B

```

Program A is destroyed.

There is nowhere to return.

What Does `exec()` Return?

This is one of the most important interview questions.

Prototype

```

C
int execl(...);

```

Return value

Condition	Return Value
Success	Does not return
Failure	-1

Why Doesn't It Return on Success?

Suppose

```
Program A
↓
exec()
↓
Program B
```

Program A disappears completely.

Therefore

```
return;
```

cannot happen.

Example Showing Return Value

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    int status;

    status = execl("/bin/abc", "abc", NULL);

    printf("Return Value = %d\n", status);

    return 0;
}
```

Output

```
Return Value = -1
```

Why?

```
/bin/abc
```

does not exist.

So

```
exec()
↓
Kernel
↓
```

File not found

↓

Return -1

Checking the Error

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    if (execl("/bin/abc", "abc", NULL) == -1)
    {
        perror("exec failed");
    }

    return 0;
}
```

Output

```
exec failed: No such file or directory
```

The Most Common Use: fork() + exec()

In Linux, `exec()` is almost always used together with `fork()`.

Program

```
C
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child before exec()\n");

        execl("/bin/ls", "ls", "-l", NULL);

        printf("This line never executes\n");
    }
    else
    {
        wait(NULL);

        printf("Parent finished\n");
    }

    return 0;
}
```

Execution Flow

```

Parent
↓
fork()
↓
+-----+
| Parent |
| PID = 1000 |
+-----+

+-----+
| Child  |
| PID = 1001 |
+-----+

↓
Child executes exec()
↓
Child Program removed
↓
Child becomes ls
↓
ls finishes
↓
Child exits
↓
Parent continues

```

Sample Output

```

Child before exec()

total 24
-rw-r--r--
drwxr-xr-x
...

Parent finished

```

Notice that:

- "Child before `exec()`" is printed because it executes **before** `exec()`.
- "This line never executes" is **never printed** because `exec()` succeeds and replaces the child process.
- The **parent process** is unaffected because only the **child** called `exec()`.

Memory Changes During `exec()`

Component	Before exec ()	After exec ()
Process ID (PID)	Same	Same
Parent PID	Same	Same
Program code (text segment)	Old program	New program
Global variables	Old	New
Heap	Old	New
Stack	Old	New
Program counter	Old location	Entry point of new program
Open file descriptors	Usually preserved unless marked close-on-exec	

Why Use fork() Before exec()?

If a process directly calls `exec()`, it loses its original program forever.

By using:

```

fork()
↓
Child created
↓
Child calls exec()
↓
Parent continues original program

```

the parent remains unchanged while the child becomes a different program. This is exactly how UNIX shells (such as `bash`) execute commands like `ls`, `pwd`, or `date`.

Final Summary

- `exec()` **does not create a new process**; it **replaces the current process image** with a new executable.
- The **PID remains the same**, but the code, stack, heap, global data, and execution point are replaced by those of the new program.
- On **success**, `exec()` **never returns** because the old program no longer exists.
- On **failure**, it returns `-1` and sets `errno`, allowing the error to be inspected with `perror()` or `strerror()`.
- The most common UNIX pattern is:

```

Parent Process
  |
  | fork()
  |

```

```

└─ Parent → continues execution
└─ Child → exec() → becomes a completely different program

```

This `fork()` → `exec()` model is the foundation of process creation and command execution in UNIX/Linux operating systems.

i couldnot understadn

No problem. This is one of the hardest concepts in UNIX. Let me explain it from **zero**, without assuming any prior knowledge.

First, Forget `exec()`

Let's first understand what happens **without** `exec()`.

Suppose we have this program:

```

C
#include <stdio.h>

int main()
{
    printf("Hello\n");
    printf("Welcome\n");
    printf("Bye\n");
    return 0;
}

```

When the CPU executes this program, it executes **line by line**.

```

main()
↓
printf("Hello")
↓
printf("Welcome")
↓
printf("Bye")
↓
return 0

```

Output

```

Hello
Welcome
Bye

```

This is normal execution.

Now imagine this program

```
C
#include <stdio.h>

int main()
{
    printf("Step 1\n");
    printf("Step 2\n");
    printf("Step 3\n");
    printf("Step 4\n");
    return 0;
}
```

Execution

```
CPU
↓
Step 1
↓
Step 2
↓
Step 3
↓
Step 4
↓
Program Ends
```

Everything is simple.

Now suppose at Step 2 we call `exec()`

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Step 1\n");
    execl("/bin/ls", "ls", NULL);
    printf("Step 3\n");
    return 0;
}
```

Now what happens?

Most students think

Step 1

↓

exec()

↓

Run ls

↓

Come back

↓

Step 3

NO!

This is completely wrong.

What actually happens?

The execution becomes

Step 1

↓

exec()

↓

Destroy THIS PROGRAM

↓

Load ls program

↓

Run ls

↓

ls finishes

↓

Program Ends

Notice carefully

Step 3

never executes.

Why?

Because **your program no longer exists.**

Imagine a notebook.

Suppose your notebook contains

```

Page 1
Hello
Page 2
Welcome
Page 3
Bye

```

Now someone says

Throw this notebook away.

You throw it away.

Then they give you a completely new notebook.

```

Linux Commands
Page 1
ls
Page 2
pwd
Page 3
date

```

Can you continue reading your old notebook?

No.

It is gone.

That is exactly what `exec()` does.

Think of Memory

Before `exec()`

```

Memory
+-----+
Our Program
main()
↓

```

```
printf()
↓
variables
↓
stack
↓
heap
+-----+
```

When `exec()` runs

The kernel says

Delete everything.

Memory becomes

Empty

Then

Load
/bin/ls

Now memory becomes

```
+-----+
ls Program
↓
its variables
↓
its stack
↓
its heap
+-----+
```

Your program is completely erased.

Very Important

Suppose your program has PID

PID = 3500

Before `exec()`

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

```
PID 3500
↓
Our Program
```

After exec()

```
PID 3500
↓
ls Program
```

Notice

Only the program changes.

The PID remains the same.

Let's See a Real Program

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("I am before exec()\n");
    execl("/bin/date","date",NULL);
    printf("I am after exec()\n");
    return 0;
}
```

CPU Execution

CPU starts

```
↓
main()
↓
printf()
```

Output

```
I am before exec()
```

Then CPU reaches

```
execl("/bin/date")
```

Immediately

Kernel deletes

main()

↓

printf()

↓

variables

↓

everything

Then kernel loads

date

Now the CPU starts executing

date program

Output

Fri Jul 4 20:30:15 IST 2026

Then

date program ends

Finished.

Did

printf("I am after exec()");

execute?

NO.

Why?

Because it was deleted.

This Is the Biggest Idea

Before

Our Program

↓

Line 1

↓
Line 2
 ↓
Line 3

After

Our Program
 ↓
exec()
 ↓
Our Program disappears
 ↓
New Program starts

Then Why Do We Use `fork()` Before `exec()`?

This is the part that makes everything click.

Suppose there is only one process.

Parent
PID 100

If this process executes

C
exec(...)

then

Parent disappears.

Sometimes we don't want that.

Instead we do

Parent
 ↓
fork()
 ↓
 +-----+
Parent
PID 100
 +-----+

```
Child
PID 101
+-----+
```

Now there are **two processes**.

Then only the child executes

```
C
exec(...)
```

```
Parent
↓
Still running
Child
↓
exec()
↓
Becomes ls
```

So

```
Parent
↓
Still our program
Child
↓
Now ls program
```

This is why UNIX shells always use:

```
fork()
↓
Child
↓
exec()
```

The shell itself (the parent) stays alive, while the child becomes the command (**ls**, **pwd**, **date**, etc.).

Return Value of `exec()`

This is another place where students get confused.

Case 1: Success

```
execl("/bin/ls", "ls", NULL);
```

The file exists.

Kernel loads `ls`.

Your program disappears.

So there is **no way to return**.

Return Value

✗ No return

Case 2: Failure

Suppose

```
C
execl("/bin/abc", "abc", NULL);
```

There is no program called `abc`.

Kernel cannot replace your program.

So your original program is still there.

Therefore `exec()` returns

```
-1
```

and execution continues.

Example:

```
C
#include <stdio.h>
#include <unistd.h>

int main()
{
    int r;

    r = execl("/bin/abc", "abc", NULL);

    printf("Return Value = %d\n", r);

    return 0;
}
```

Output

```
Return Value = -1
```

because `/bin/abc` does not exist.

One-Line Rule to Remember

`fork()` creates a new process.

`exec()` does not create a process—it replaces the program running inside the current process.

I can also explain `fork()` and `exec()` together using a memory animation (step-by-step diagrams showing what happens inside RAM and the Process Control Block). That visualization usually makes the concept click in 5 minutes.